

Beyond ReAct: Evaluating and Improving LLM Tool Agents on Real-World Benchmarks

Muntaha Asim
muntahaasim79@gmail.com

Muhammad Ammar Kashif
ammarkashif00@gmail.com

Abstract—The automation of machine learning (ML) experimentation has long been a goal of the artificial intelligence research community. Recent advances in large language models (LLMs) have opened new possibilities for building autonomous agents capable of performing complex, multi-step tasks such as writing and executing code, analyzing results, and iterating toward improved model performance. This paper reviews the current state of LLM-based agentic systems applied to ML experimentation, with a focus on benchmarking, agent design, and key limitations. We examine foundational frameworks such as ReAct and Reflexion, survey benchmark efforts including MLAgentBench and AgentBench, and analyze the performance of agents powered by models such as GPT-4, Claude, and Gemini across diverse ML tasks. Despite promising results, significant challenges remain, including hallucination, poor long-term planning, and limited generalization to tasks unseen during model pretraining. This review identifies these open problems and outlines directions for future research toward more reliable and capable ML experimentation agents.

Index Terms—large language models, autonomous agents, machine learning experimentation, benchmarking, ReAct, MLAgentBench, AutoML

I. INTRODUCTION

Machine learning research is fundamentally driven by experimentation. A typical ML workflow involves formulating a hypothesis, selecting an appropriate model architecture, implementing and executing training code, evaluating performance metrics, and iterating based on the results. This process demands not only deep technical knowledge but also the ability to reason across long time horizons, debug code, and draw meaningful conclusions from empirical observations. Traditionally, this workflow has been carried out entirely by human researchers, making it both time-intensive and expertise-dependent.

The emergence of large language models (LLMs) such as GPT-4 [1], Claude [2], and Gemini [3] has introduced a fundamentally new paradigm in AI. These models exhibit remarkable capabilities in natural language understanding, code generation, and structured reasoning. Building on these capabilities, recent research has explored the possibility of constructing autonomous agents — systems that can perceive their environment, plan sequences of actions, and execute those actions with minimal human intervention [4], [5].

LLM-based agents represent a promising avenue for automating ML experimentation end-to-end. Rather than relying on narrowly scoped AutoML pipelines that optimize a fixed set of hyperparameters, agentic systems can interact with a file

system, write and modify training scripts, execute code, and adaptively revise their strategies based on observed outcomes. This flexibility allows them to operate in open-ended settings that more closely resemble real-world research workflows.

However, existing agentic systems face significant limitations when applied to complex ML tasks. Key challenges include the tendency to hallucinate results — claiming performance improvements without empirical verification — poor planning over extended interaction sequences, and weak generalization to problem domains that were not represented in the model’s pretraining data [6]. These limitations reduce the reliability of LLM-based agents in settings where correctness and reproducibility are critical.

This paper reviews the current landscape of LLM-based agents for ML experimentation. We examine the design of agent frameworks, the benchmarks used to evaluate them, and the empirical performance reported across a range of tasks and models. By synthesizing findings from recent literature, we aim to characterize the state of the field, identify persistent gaps, and motivate future research directions.

II. LITERATURE REVIEW

A. LLM-Based Agent Frameworks

The development of general-purpose LLM agents has been an active area of research. Yao et al. introduced ReAct [7], a framework that interleaves reasoning traces with action execution, enabling agents to maintain a coherent internal state while interacting with external environments. ReAct demonstrated that prompting LLMs to reason before acting significantly improves task completion across question answering and interactive decision-making benchmarks. This design principle has since been widely adopted in subsequent agentic systems.

Shinn et al. proposed Reflexion [8], which extends ReAct by introducing verbal reinforcement learning. In Reflexion, agents generate natural language reflections on their previous failures and use these reflections to improve future behavior without updating model weights. This approach proved particularly effective in tasks requiring iterative refinement, making it directly relevant to ML experimentation workflows where agents must learn from failed experiments.

Park et al. explored generative agents [5] that simulate believable human behavior through memory, reflection, and planning modules. While not directly applied to ML experimentation, their architecture demonstrated that LLMs can

maintain coherent long-term behavior across extended interaction sequences — a key capability required for autonomous research workflows.

B. Benchmarking Agentic Capabilities

Evaluating the capabilities of LLM-based agents requires carefully constructed benchmarks that capture the complexity of real-world tasks. Liu et al. proposed AgentBench [9], a comprehensive evaluation framework that tests LLMs across eight distinct environments including web browsing, database interaction, and operating system tasks. AgentBench revealed substantial performance gaps between different model families and highlighted the difficulty of sustaining consistent reasoning across long task horizons.

Zhou et al. introduced WebArena [10], a realistic web-based environment for evaluating agents on tasks that require navigation, form submission, and information retrieval across live web applications. Their findings showed that even state-of-the-art LLMs achieve relatively low success rates on realistic web tasks, suggesting that agentic performance degrades significantly as task complexity increases.

Kinniment et al. proposed ARA [11], a benchmark targeting high-stakes autonomous tasks that require extended planning and execution. ARA emphasizes the importance of safety and reliability in agentic systems, particularly when agents are given access to real computational resources.

C. Agents for Machine Learning Experimentation

Huang et al. introduced MLAGentBench [6], the first benchmark specifically designed to evaluate LLM-based agents on ML experimentation tasks. MLAGentBench comprises 13 tasks spanning diverse domains including image classification, natural language processing, graph learning, and code optimization. For each task, an agent is provided with starter code, dataset files, and a task description, and must iteratively improve model performance through file editing and code execution.

Experiments in MLAGentBench evaluated agents powered by GPT-4, GPT-4-turbo, Claude v1.0, Claude v2.1, Claude v3 Opus, Gemini Pro, and Mixtral. Claude v3 Opus achieved the highest average success rate of 37.5%, while other models performed considerably lower. Importantly, success rates dropped sharply on tasks introduced after the models’ pretraining cutoff dates, with some Kaggle challenges yielding 0% success across all models [6]. This finding highlights a critical limitation: agents relying solely on pretrained knowledge struggle to generalize to genuinely novel problem settings.

The agent design in MLAGentBench incorporates structured reasoning through four components: *Reflection*, *Research Plan and Status*, *Fact Check*, and *Thought*. The Fact Check component was introduced specifically to mitigate hallucination — a prevalent failure mode in which agents falsely claim performance improvements without executing the relevant code. While this design reduced hallucination frequency, it did not eliminate it entirely, indicating that structural prompting alone is insufficient for reliable agentic behavior.

D. AutoML and LLM-Assisted Optimization

Prior to the emergence of fully agentic systems, the ML community explored AutoML as a means of reducing human effort in model development. Neural Architecture Search (NAS) [12] automates the design of neural network architectures, while hyperparameter optimization frameworks such as AutoML [13] search over predefined configuration spaces. These approaches, however, are constrained to fixed search spaces and lack the flexibility to reason about arbitrary code or adapt to novel problem formulations.

Zhang et al. proposed AutoML-GPT [14], which prompts LLMs with data and model cards to predict training outcomes and guide hyperparameter selection. While demonstrating the utility of LLMs for structured optimization tasks, AutoML-GPT operates within a narrower scope compared to fully interactive agentic systems. Similarly, MLCopilot [15] uses LLMs to recommend categorized hyperparameter settings based on accumulated past experience, but does not support open-ended code interaction. These approaches collectively suggest a continuum from constrained AutoML pipelines toward fully autonomous agentic systems, with LLM-based agents representing the open-ended end of this continuum.

E. Identified Gaps and Open Problems

Despite notable progress, several important gaps remain in the literature. First, existing benchmarks primarily evaluate task success rates without deeply analyzing the quality of the agent’s reasoning process. Second, most current work focuses on single-agent architectures, leaving multi-agent collaboration for ML experimentation largely unexplored. Third, the impact of context window limitations on long-horizon task performance has not been systematically studied. Finally, the safety implications of agents that autonomously write and execute code on real systems remain underexplored, despite being directly relevant to deployment in research environments. Addressing these gaps represents a significant opportunity for future work in agentic AI.

REFERENCES

- [1] OpenAI, “GPT-4 technical report,” *arXiv*, abs/2303.08774, 2023.
- [2] Anthropic, “Introducing Claude,” 2023. [Online]. Available: <https://www.anthropic.com/index/introducing-claude>
- [3] G. T. G. R. Anil et al., “Gemini: A family of highly capable multimodal models,” *arXiv*, abs/2312.11805, 2023.
- [4] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. J. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *arXiv*, abs/2305.16291, 2023.
- [5] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [6] Q. Huang, J. Vora, P. Liang, and J. Leskovec, “MLAgentBench: Evaluating language agents on machine learning experimentation,” in *Proc. 41st International Conference on Machine Learning (ICML)*, PMLR 235, 2024.
- [7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [8] N. Shinn, F. Cassano, A. Gopinath, K. R. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*, 2023.

- [9] X. Liu et al., “AgentBench: Evaluating LLMs as agents,” in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [10] S. Zhou et al., “WebArena: A realistic web environment for building autonomous agents,” *arXiv*, abs/2307.13854, 2023.
- [11] M. Kinniment et al., “Evaluating language-model agents on realistic autonomous tasks,” *arXiv*, abs/2312.11671, 2023.
- [12] T. Elsken, J. H. Metzen, and F. Hutter, “Neural architecture search: A survey,” *Journal of Machine Learning Research*, vol. 20, no. 1, pp. 1997–2017, 2019.
- [13] X. He, K. Zhao, and X. Chu, “AutoML: A survey of the state-of-the-art,” *Knowledge-Based Systems*, vol. 212, p. 106622, 2021.
- [14] S. Zhang, C. Gong, L. Wu, X. Liu, and M. Zhou, “AutoML-GPT: Automatic machine learning with GPT,” *arXiv*, abs/2305.02499, 2023.
- [15] L. Zhang, Y. Zhang, K. Ren, D. Li, and Y. Yang, “MLcopilot: Unleashing the power of large language models in solving machine learning tasks,” *arXiv*, abs/2304.14979, 2023.